

TEAM BERRY:

- Blaž Boček - programiranje, audio in art
- Matic Jurač - programiranje, AI in user controls
- Miha Bravc – programiranje, uporabniški vmesniki

IME IGRE: Winter March

KONCEPT: Cilj igre je priti do konca naključno generiranega zemljevida, kjer vsako vozlišče začne boj, podobnemu šahu in doda nove figure na igralno polje. Igralec premika figure da požre nasprotnikove pošasti, a po vsakem boju vedno več polj zakrije “snow of war”, t.j. sneg, ki omejuje gibanje in vidljivost igralcu. Igralec mora vsak boj vzeti vse pošasti, in pri tem ne izgubiti svojih likov, med boji pa pripravi svojo ekipo na vedno bolj zakrito mapo, torej zimo, ki prihaja.

KLJUČNE MEHANIKE

- Premikanje figur in zajemanje sovražnikovih
- Turn-based boji proti AI
- Rogue-like zemljevid
- Fog of war
- Dodajanje figur k sovražniku in sebi

ORODJA: Godot, Pixel Studio, Audacity, GIMP

POUDARKI: Šahu podobna gameplay zanka z fog of war in spremenljivimi figurami, horror in strateški elementi z vedno večjo prekritostjo mape, preprost AI, naključno generiran zemljevid.

VIRI: Vsi vizualni in zvočni asseti, ki se pojavijo v igri, so originalni. LLM-i so se uporabljali izključno kot kodna asistenca in orodje za razhroščevanje.

STATUS:

- Dokončano: Naključno generiran zemljevid, ki spreminja persistentno sovražno in igralčevo ekipo, boji z naključno generiranimi ovirami, sovražnim AI in fog of war mehanikami, osnovna gameplay zanka in boji.
- Nedokončano: Imeli smo idejo, da bi naredili, da so nasprotnikove figure lahko začarane, in da se party ureja kot v ostalih rogue-like igrah, in da se figure lahko nadgrajajo tako, da kupiš nadgradnje v trgovinah z valutami kot na primer usnje, ki bi dobil po zmagani tekmi. Torej bi še dodali trgovine in kamp, kjer bi lahko igralec svoj party spočil in zamenjal figure. Imeli smo tudi idejo, da bi igralčev kralj bil nek vir toplote in bi spraznil veliko snega, ampak smo ga zaradi časovne stiske spremenili v damo z dodanimi potezami viteza.

RAZVOJNI PROCES

Igro smo načrtovali kvečjemu v prvemu dnevu, vsi smo predstavili naše ideje in na papir narisali "osnutke". Idejo igre smo spreminjali sproti, ko so se časovne omejitve tekmovanja gamejam začele prikazovati. Dela si nismo načeloma pred začetkom razdelili, vsak je naredil to, pri čemer se je počutil najbolj samozavesten. Vsi smo delali z Godot-om in orodjem za skripting, ki je dodan, Blaž, ki je naredil audio in video je še uporabljal Pixel Studio, GIMP in Audacity.

Načrtovanje, Koncept in Iteracija

1. Od ideje do končnega koncepta

Izhodišče projekta je bila tema "Winter Is Coming". Ekipa je sprva razmišljala o sistemu za upravljanje virov in glavno zanko, kjer bi se igralec pripravljala na sovražnike in se branil. Že v tej fazi je bila določena uporaba mrežnega sistema in bojnega sistema.

Kasneje smo izvedli večkratno zmanjševanje obsega

- Namesto odprtega sveta se je koncept preusmeril v Rouge-based igro z napredovanjem po svetu, kjer se težavnost in zima stopnjujeta z vsakim korakom.
- Končna odločitev je bil šahovski stil igre, kjer igralec napreduje po vozliščih. Vsako obiskano vozlišče poveča število figur, ki jih igralec lahko uporabi, kar se zrcali tudi pri sovražnikih.

2. Vizualizacija napredka (Snow of War)

Ključni element napredka je Megla vojne (Snow of War), ki pokriva mapo in se postopoma odmika z napredovanjem igralca. Odločili smo se da je snega vedno več slojev od zgoraj navzdol, vendar ne na zadnjih dveh nivojih. In tudi dlje kot igralec gre v igro več figur ima on in tudi nasprotnik.

3. Metodologija načrtovanja

Ekipa se je zanašala na mešanico pogovorov, risanja na papir in na tablični računalnik za takojšnje usklajevanje idej in tehnične izvedbe. Potem, ko smo pa že implementirali igro smo pa pri obrokih ustno se dogovarjali, kaj bomo obdržali in/ali ne.

Organizacija Dela in Timsko Delo

- **Delitev vlog:** Razdelitev je potekala na podlagi hitrega dogovora, kjer je vsak član enostavno prevzel svoj segment dela
- **Komunikacija in Sinhronizacija:** Vsi smo bili skupaj v isti hiši, vendar smo občasno uporabili **Discord** za komunikacijo, in za sinhronizacijo kode smo uporabili **GitHub**.
- **Upravljanje s kodo:** Združevanje kode je potekalo s pomočjo **Gita**.

Orodja in Tehnike Razvoja

Asseti so bili izdelani s sledečimi orodji:

Element	Orodje	Vloga v razvoju
Igralni Engine	Godot Engine 4.x	Centralno razvojno okolje. Vsa koda je bila napisana znotraj vgrajenega urejevalnika Godot
Verzija Krmiljenje	GitHub / GitKraken	GitHub za shranjevanje repozitorija; GitKraken kot grafični vmesnik za lažje upravljanje Git vej in združevanje sprememb.
Grafika (Pixel Art)	Pixel Studio	Uporabljen za ustvarjanje vseh <i>Pixel Art</i> assetov
Zvok	Audacity	Uporabljen za snemanje in kompozicijo zvočnih efektov
Slike	GIMP	Modificirane fotografije.

2. Podrobnosti o Mehanikah in Sistemih

Logika igre je zgrajena na dveh povezanih plasteh: Strateška plast (navigacija po mapi sveta) in Taktična plast (boj na mreži).

Strateška Plast: Navigacija, Napredek in Nadzor Kamere

Sistem mape, ki ga upravlja MapController.gd, skrbi za progresivno razkrivanje sveta in interakcijo igralca z vozlišči.

1. Dinamični Sistem Snow of War (SOW) in Napredek

- **Povezava z nadstropjem:** SOW je vezan na lastnost `PlayerManager.current_map_floor`, ki predstavlja najvišjo doseženo Y-koordinato na zemljevidu poti.
- **Logika Nepovratnega Napredovanja:** V `PlayerManagerju` je implementirana varnostna logika, ki preprečuje regresijo. Kadarkoli se izbere nova soba, se novo nadstropje (Y-koordinata) shrani samo, če je višje od trenutno shranjene vrednosti.
- **Implementacija v GridManagerju:** `GridManager.initialize_all_fog()` izračuna število vrstic, ki naj jih megla pokrije, na podlagi trenutnega nadstropja. Vsa polja `TileMap`-a nad izračunano mejo (`fog_limit_y`) so pokrita z dinamičnimi vozlišči megle, ki se odstranijo takoj, ko igralec napreduje.

2. Kontrola Kamere in Omejevanje (Clamping)

MapController v funkciji `_input(event)` zagotavlja omejeno premikanje kamere.

- **Dvojni mehanizem omejevanja:**
 - **Premikanje (Pan):** Medtem ko igralec premika miško (`InputEventMouseMotion`), se izračuna nova pozicija kamere, ki se nato omeji znotraj vnaprej določenih meja mape.
 - **Povečava (Zoom):** Ker se pri povečavi (`MOUSE_BUTTON_WHEEL`) spremeni velikost vidnega polja (`half_viewport`), se z vsako spremembo zooma ponovno izračunajo točne meje omejitve in kamera se takoj premakne na veljavno pozicijo.

Taktična Plast: Mrežna Logika in Karakterji

Ko igralec vstopi v bojno sobo, se aktivira `GridManager`, ki služi kot centralni register bojnega polja.

1. Mrežni Koordinatni Sistem in Zasedenost

- **Grid Manager kot register:** `GridManager` je odgovoren za inicializacijo polja in vzdrževanje globalnega stanja zasedenosti. Vzdržuje slovar **occupied**, ki beleži, kje se nahaja katera figura. To omogoča hitro iskanje tarč (`get_character_at(grid_pos)`) in preprečuje kolizije (dve figuri ne moreta stati na istem mestu).
- **Inicializacija bojnega polja:** Ob zagonu dogodka `GridManager` registrira vse figure igralca in sovražnika na določene mrežne koordinate in jim dodeli reference nase.

2. Dinamično Poudarjanje Premikov in Zajetja

Vizualni podsistem za premikanje preklopi barvo poudarjanja glede na ciljno polje.

- **Logika MoveHighlighterja:** Poudarjanje se izvaja v funkciji `_draw()`. Za vsako veljavno mreženo pozicijo, ki jo figura lahko doseže, se izvede preverjanje:
 - **Zeleno (Premik):** Če je polje prazno ali zasedeno s prijateljsko figuro (kar v naši logiki pomeni veljaven premik na prazno polje), je barva zelena (`MOVE_COLOR`).
 - **Rdeče (Zajetje):** Če klic `grid_manager.get_character_at(grid_pos)` vrne nasprotnikovo figuro, se barva preklopi na rdečo (`CAPTURE_COLOR`), kar signalizira da lahko napadeš nasprotnika.

Obnašanje Sovražnika (Enemy AI)

Logika sovražnikov se nahaja v BaseCharacter.gd in je zgrajena na sistemu stanj.

- **Zaznavanje in Sledenje:** Ko sovražnik figuro igralca zazna, se aktivira spremenljivka `has_spotted_player`, in posodobi se lokacija **`last_known_player_pos`**. Sovražnik nato uporabi to lokacijo za določitev smeri svojega premika (`last_known_direction`).
- **Stanje Panike:** Za dodajanje elementa taktične negotovosti je vpeljeno stanje panike (`is_panicking`). Če se sovražnik znajde znotraj **`panic_distance`** od določenih nevarnih situacij, se preklopi na kaotično obnašanje. Namesto da bi sledil izračunani poti, se njegovo gibanje določi naključno v mejah, določenih s parametrom `panic_randomness`.

3. Tehnična Izvedba in Arhitektura

Programska Struktura in Arhitektura Projekta

Glavne strukture projekta:

Razred/Vzorec	Namen	Ključne Odgovornosti
Autoloads	Globalno upravljanje stanja in podatkov.	PlayerManager (globalno stanje igralca, inventar, napredek FOW), UiAudio (upravljanje zvočnih efektov).
MapController.gd	Upravljanje strateške plati igre (nad svetovno mapo).	Obdelava vnosa kamere, omejevanje pogleda (clamp), vizualizacija sob in posredovanje interakcij PlayerManagerju.
GridManager.gd	Upravljanje taktične plati igre (bojna mreža).	Register zasedenosti polj (occupied), koordinatni sistem, inicializacija figure, posodabljanje stanja <i>Fog of War</i> .
BaseCharacter.gd	Abstraktni lik/enota.	Osnovne lastnosti figure (doseg, pozicija), logika AI (zaznavanje, panika), zvočni efekti (premiki).
Room.gd	Data-Resource za sobo.	Shrani metapodatke vozlišča (povezave do naslednjih sob, tip sobe, stanje odklenjenosti/izbranosti).

Ključni Tehnični Izzivi in Rešitve

Razvoj je zahteval reševanje več tehničnih izzivov, zlasti pri delu z Godot 4 API-jem in kombinacijo proceduralnih in taktičnih sistemov.

1. Dinamično Omejevanje Kamere

- **Izziv:** Ohraniti kamero znotraj meja proceduralno generirane mape, medtem ko je omogočena spremenljiva povečava (*zoom*).
- **Rešitev:** V MapController se meje mape izračunajo enkrat (`_calculate_map_boundaries`). Pri premikanju in povečavi se nato meje za center kamere (`clamp_min/clamp_max`) dinamično prilagajajo. Ta prilagoditev upošteva polovico velikosti vidnega polja, deljeno s trenutnim zoomom (`half_viewport = get_viewport_rect().size / map_camera.zoom / 2.0`).

2. Nezvezno Povečevanje Nadstropij (Fog of War Progress)

- **Izziv:** Ker proceduralna generacija ne zagotavlja, da je vsako naslednje vozlišče strogo v spodnjem nadstropju, bi lahko navigacija nazaj povzročila zmanjšanje FOW.
- **Rešitev:** V PlayerManager je bila v funkcijo `set_current_floor()` vgrajena logika, ki uporablja funkcijo **max()**. Nova vrednost se shrani samo, če je večja od trenutne vrednosti `current_map_floor`.

3. Ločevanje Poudarjanja in Figur (Move Highlighter)

- **Izziv:** Učinkovito vizualno poudarjanje veljavnih premikov in tarč, pri tem pa zagotoviti, da poudarek spada k igralnemu polju in ne k figuri.
- **Rešitev:** Namesto da bi poudarek risala figura sama, je bila uvedena ločena komponenta MoveHighlighter. Ta komponenta deluje kot vizualni sloj in uporablja `_draw()` za risanje pravokotnikov na podlagi koordinat v **valid_moves**. MoveHighlighter se poveže z GridManagerjem in v realnem času preverja, ali je polje prazno (zeleno poudarjanje) ali zasedeno z nasprotnikom (rdeče poudarjanje za zajetje).

Uporabljeni Sistemi

Proceduralna Generacija

- Mapa sveta se ustvari s skripto MapGenerator.gd, ki ustvarja strukturiran graf vozlišč (Room) z navpično hierarhijo, ki simulira nivoje.

AI in Obnašanje Figur

- AI v BaseCharacter.gd je zasnovan na vektorski logiki sledenja in stanju panike. Sovražnik sledi zadnji znani poziciji igralca. Če se sproži stanje panike (is_panicking), se gibanje preklopi na naključno smer, ki se v določenih mejah izbere okoli izračunane ciljne smeri.

Koordinatni Sistem in Fizika

- Gibanje in interakcije so zasnovane na mrežnem koordinatnem sistemu (Vector2i). GridManager in BaseCharacter izvajata vse pretvorbe med mrežnimi koordinatami (za logiko) in svetovnimi koordinatami (za vizualizacijo).

4. Vizualni in zvočni del

Vsi grafični in zvočni elementi v projektu smo sami naredli. Nismo uporabili nobenih zunanjih asset packov ali orodij umetne inteligence za generiranje vizualne ali zvočne vsebine.

Umetniški slog je pixel art s fantastičnimi elementi.

Ustvarjanje Ozadij in Ploščic (Tiles)

Slog je bil dosežen s mešanico fotografije in ročne digitalne obdelave:

1. **Ozadje glavnega menija:** Izvorna slika je bila originalna fotografija gozda.
2. **Igralna mreža:** Tla so sestavljena iz dveh kontrastnih ploščic. Za ustvarjanje tekstur sta bili uporabljeni dve lastni fotografiji tal (ena s snegom in druga s temno travo). Obe fotografiji sta bili nato pixelizirani, obdelani in prilagojeni, da delujejo kot tiles na igralni mreži.

Ustvarjanje Figur (Sprites)

Vse igralne figure (Pawn, Knight, Bishop, Rook itd.) so bile ročno narisane v pixel art stilu. Ta grafični slog je bil usklajen z ločljivostjo in barvno paleto pixeliziranih ozadij, kar zagotavlja vizualno povezavo med liki in okoljem.

Zvočni Posnetki, Glasba in Efekti

Zvočni Efekti: zvočni efekti so bili ročno posneti in nato digitalno obdelani

5. Namestitev in Zagon Igre

Struktura Repozitorija in Ključne Datoteke

- **Mapa projekta:** Celotna izvorna koda Godot projekta se nahaja v korenski mapi repozitorija.
- **Datoteka Projekta:** Glavna konfiguracijska datoteka Godot je **project.godot**, ki se nahaja v korenskem direktoriju.
- **Izvorna koda:** Večina logike se nahaja v mapi `res://Scripts/`
- **Asseti:** Vse assete in zvočni posnetki se nahajajo v mapi `res://Assets/`

Zagon Izvožene Igre (Izvedljiva datoteka)

Igra je izvožena za okolje Linux.

- **Zagonska datoteka:** Izvedljiva datoteka se nahaja v korenskem direktoriju in se imenuje `winter_march.elf` (datoteka je v formatu ELF).
- **Zahtevani Runtimes:** Za zagon ELF datoteke ni potrebnih nobenih dodatnih datotek ali Runtimov, saj je Godot Engine vgrajen v izvoz. Vendar pa je za zagon v Linuxu morda potrebno nastaviti dovoljenje za izvajanje.

Postopek Zagon (Linux):

1. Dovoljenje za izvajanje: Odprite terminal v direktoriju, kjer se nahaja datoteka `.elf`.
2. Nastavite dovoljenje za izvajanje z ukazom:

`chmod +x [IME DATOTEKE]`

Zagon iz Godot Editor

Ta metoda omogoča vpogled v celotno strukturo projekta in kodo.

- Zahtevano orodje: Za zagon iz vira je nujna namestitev Godot Engine verzije 4.x
- Postopek:
 - Prenesite celoten repozitorij.
 - Odprite Godot Engine 4.x.
 - Izberite Uvozi/Import in izberite mapo projekta, ki vsebuje datoteko `project.godot`.
 - Odprite projekt v urejevalniku in pritisnite tipko F5 ali ikono Zaženi/Run na vrhu urejevalnika

6. Uporaba AI in Zunanjih Virov

Orodja umetne inteligence niso bila uporabljena za generiranje kreativne vsebine (grafike in zvočnih efektov). Vsi vizualni in zvočni elementi so originalno avtorsko delo. AI je bila uporabljena kot asistent za kodiranje in debugging, kar je prispevalo k hitrosti razvoja.

V te namene smo uporabili modele:

- Gemini 2.5
- ChatGPT 3.5

Izbrali smo te modele saj so brezplačni in imajo dobre funkcionalnosti za namen asistenta pri kodiranju in debugging.